
Training Convolutional Networks with Spatial Continuation

Aubin Charley

Alexandre Corrad

Idriss El khamlichi

Maxime Nicaise

CentraleSupélec, Université Paris-Saclay

Abstract

An idea to train a neural network is to start from a simpler problem and progressively recover the original one. We investigate this idea by simplifying the spatial information inside convolutional networks during training. We compare temporary resolution reduction, Gaussian filtering of feature maps, and their combination, all ending at the original inference architecture. An exploratory CIFAR-10 study selects three procedures, which we then examine across datasets, architectures and optimizers. Resolution reduction improves mean test accuracy in all twelve benchmark settings. The additional benefit of filtering depends on the training recipe. On STL-10, the combination performs below the baseline although either intervention alone improves it. We also examine the loss landscape around the trained models on CIFAR-10. Across five seeds, all three methods have smaller largest Hessian eigenvalues than the baseline with fixed BatchNorm statistics. However, Gaussian filtering alone shows a larger increase in test loss under weight perturbations when these statistics are recalibrated. The accuracy gains therefore do not consistently coincide with lower sensitivity across these measurements.

1 Introduction

An idea to minimize a difficult objective is to start from a simpler approximation and progressively recover the target problem. Continuation methods follow this principle by linking problems through a parameterized family and using the previous solution to initialize the next stage (Allgower and Georg, 1990; Mobahi and Fisher, 2015). Graduated optimization provides guarantees under specific geometric assumptions (Hazan et al., 2016). Constructing useful families for neural-network training remains less straightforward.

Our starting point was the Gaussian filtering of CNN feature maps proposed by Sinha et al. (2020). The intuition is that the model can first focus on large-scale structures before dealing with finer details and textures. We then viewed the Gaussian kernel used in this reference more generally as a way of controlling spatial information. This led us to temporarily reduce the resolution in the internal layers of the network. The question became: how much of the observed benefit comes from Gaussian smoothing, and how much can be obtained by simply processing smaller feature maps early in training?

We used a ResNet-20 on CIFAR-10 to explore operators, insertion points and schedules, and to select three procedures: resolution reduction only, Gaussian filtering only (from Sinha et al. (2020)), and their combination. The exploration is reported separately in Appendix B. We then kept these procedures fixed, with the adaptations required by the new settings, and evaluated their transfer on other datasets, other architectures and optimizers. Resolution reduction gives the most consistent accuracy gains, while the usefulness of the Gaussian depends on the optimizer and dataset. Finally, we examine the trained models through weight perturbations, Hessian calculations and input derivatives. These measurements narrow the possible interpretations: improved test performance does not require the same ordering of local sensitivities, and the treatment of BatchNorm changes what a loss-landscape comparison measures.

2 Related work

Continuation and curricula. Classical continuation methods trace solutions through parameterized problems (Allgower and Georg, 1990). For nonconvex optimization, Mobahi and Fisher (2015) analyze Gaussian-smoothed objectives, while Hazan et al. (2016) identify a class for which graduated optimization converges globally. Spatial filtering inside a network is not equivalent to smoothing its loss in parameter space, so these guarantees do not directly apply here. Related approaches include noising the activations and

deforming the network in Mollifying Networks (Gulcehre et al., 2016). Curriculum learning progressively changes which examples contribute most to the loss (Bengio et al., 2009). Our methods keep the examples and labels fixed and change their spatial processing.

Spatial information during training. Curriculum by Smoothing (CBS) (Sinha et al., 2020) is our direct starting point: Gaussian kernels filter feature maps, and their scale decreases during training. Other approaches rely on change the resolution of the input images during training. For instance, Hoffer et al. (2019) use different image sizes, while Tan and Le (2021) progressively increase the resolution and adjust the regularization accordingly. SDPoint (Kuen et al., 2018) also reduces the resolution of internal feature maps. At each training iteration, it randomly selects a block boundary and a reduction ratio, then applies average pooling. Our procedure instead uses max-pooling at fixed locations and progressively reverts the original resolution. While SDPoint allows the inference cost to be adjusted by choosing a downsampling configuration, our modification is temporary and ends at the original inference architecture.

Frequency and aliasing. Rahaman et al. (2019) describe a bias towards low-frequency functions during neural-network learning. In this setting, frequency refers to variations of the learned function over its input space, which is different from the spatial frequencies of the feature maps we filter. For these feature maps, reducing resolution and removing high frequencies are also different things, and their order matters. Applying a low-pass filter before subsampling can make predictions more stable under small image changes (Zhang, 2019). The choice of where to insert the filters affects generalization (Vasconcelos et al., 2021). The picture is further complicated by nonlinearities, which can introduce new frequency components and aliasing (Michaeli et al., 2023). These distinctions matter for our Gaussian interventions: placing a blur after a strided convolution does not undo aliasing that has already occurred in that convolution.

Geometry and sensitivity. Our procedures recover the original architecture at the end of training, but follow different optimization paths. To examine the resulting models, we draw on the loss visualizations of Li et al. (2018), using filter-normalized directions and trajectory projections. These plots require care: reparameterizing a network can change its apparent sharpness without changing its predictions (Dinh et al., 2017). We also examine local curvature through Hessian-vector products, which avoid constructing the full Hessian (Pearlmutter, 1994; Ghorbani et al., 2019). Alongside

these weight-space measurements, input Jacobians describe how network outputs respond to changes in the images. They also appear in generalization bounds based on classification margins (Sokolić et al., 2017). Here, we use these tools to compare the trained models, but don’t claim that the measured differences explain their accuracy gains.

3 Spatial continuation

Let $\mathcal{D} = (x_i, y_i)_{i=1}^n$ be the training sample and f_θ the original network. During training, we modify the spatial resolution and, when filtering is used, the filter scales of the inner layers. We describe these choices by a state s and write $f_{\theta,s}$ for the corresponding network, with loss

$$L_s(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta,s}(x_i), y_i), \quad f_{\theta,s_*} = f_\theta, \quad (1)$$

where ℓ is cross-entropy and the final state s_* restores the original network. We train for a fixed number of stochastic gradient updates at each state, then move to the next while keeping the weights and optimizer state. This gradually recovers the original training problem without requiring each intermediate problem to be solved exactly. We use “continuation” to describe this progression. Since resolution changes are discrete and max-pooling is nonsmooth, we do not claim to follow a smooth path of minimizers.

3.1 Resolution reduction

Here we reduce the spatial size of a feature map $h \in \mathbb{R}^{C \times H \times W}$ using adaptive max-pooling, which keeps the largest value in each window, separately for each channel:

$$[R_{r_h, r_w}(h)]_{c,i,j} = \max_{(p,q) \in B_{i,j}} h_{c,p,q}. \quad (2)$$

Each window $B_{i,j}$ contains the positions of the original map summarized by the output value at (i, j) . For example, reducing a 32×32 map to 16×16 amounts to taking the maximum in each 2×2 block. For other size changes, the windows can overlap their exact definition is given in Appendix A.3. In the selected ResNet-20, we apply this operation to the full output of the second residual block, after the residual addition and final ReLU. The resulting map feeds both branches of the next block. Its side length increases from 16 during epochs 0–5 to 24 during epochs 6–11, then returns to 32 for epochs 12–29.

The following layers process these smaller maps directly, without first upsampling them. Global average pooling ensures that the classifier still receives an input of the

Table 1: Selected procedures in ResNet-20. R, G and RG are used as abbreviations in tables. Plain has no intervention.

Method	Reduction	Added Gaussian
R	After block 1	None
G	None	10 post-ReLU sites
RG	After block 1	19 conv outputs

same dimension, and all trainable parameter shapes remain unchanged. Although this operation reduces spatial resolution, it does so by retaining local maxima and is therefore not a linear low-pass projection.

3.2 Gaussian filtering

For these methods, we smooth each feature map h_j by averaging neighboring values with Gaussian weights as follows

$$h_{j,s} = G_{\sigma_j(s)} * h_j, \quad (3)$$

where $\sigma_j(s)$ controls the amount of smoothing. The filter is applied independently to each channel. The filter introduces no trainable parameters, and setting $\sigma_j(s) = 0$ leaves the feature map unchanged. In the reference ResNet-20, the Gaussian-only method applies the filter after the initial ReLU and after the final ReLU of each residual block, giving ten filtering sites. The combined method instead filters the outputs of all 19 convolutions, before BatchNorm.

Both methods use a base scale $g(e)$ that decreases from 1 to 0.3 in stages lasting three epochs each, with filtering removed from epoch 21 onward. For Gaussian-only, the filter scale is simply $g(e)$; in the combined method, it is also adjusted to the resolution schedule, as described below.

3.3 The three selected procedures

From our CIFAR-10 exploration (Appendix B), we retain three procedures: resolution reduction alone, which we call RESOLUTION (R); Gaussian filtering alone, GAUSSIAN (G); and their combination, COMBINED (RG). Table 1 summarizes their settings. We compare them with Plain, the original network trained without either intervention.

The combined procedure follows the same resolution schedule as R, with one additional rule linking the Gaussian scale to the resolution. If $r(e)$ is the side length chosen at epoch e , its filter scale is

$$\sigma_j(e) = g(e) \frac{r(e)}{32} \quad \text{at all 19 filtering sites.} \quad (4)$$

The same factor is applied even at the five sites before the reduction, where the maps remain 32×32 . As

a result, an increase in resolution can temporarily increase the effective blur, even though $g(e)$ decreases (Appendix A.4).

Together with the different filtering locations described above, this means that RG does more than simply add the Gaussian-only procedure to R. We therefore compare three complete training procedures, rather than independently switching the same two interventions on and off. All three end at the original resolution with the added operations disabled.

4 Experimental study

4.1 Selection and evaluation

The starting exploratory setting uses CIFAR-10 (Krizhevsky, 2009), ResNet-20 (He et al., 2016) with BatchNorm (Ioffe and Szegedy, 2015), and 30 epochs of SGD without augmentation. We use all 50,000 training images and the 10,000 test images. As a final step in the exploration, we compared 16 configurations, each trained with three random seeds. For each seed, all configurations shared the same initial weights and training-example order. This comparison helped us choose the three procedures described in Section 3. R reaches $80.37 \pm 0.61\%$ test accuracy and RG $81.51 \pm 0.22\%$, against $75.43 \pm 0.76\%$ for plain. Appendix B gives the comparisons and failures that led to these choices.

We evaluate each model after the final scheduled epoch, not the epoch with the highest test accuracy. Evaluation uses the original network with the added operations disabled and, for BatchNorm, the saved running statistics. For each experimental setting, we report the mean and sample standard deviation across training seeds. To measure a gain over Plain, we first subtract the Plain result for the same seed, with matching initial weights and training-example order. We then report the mean and standard deviation of these differences. Results from separate experimental campaigns are kept separate.

4.2 Transfer of the selected procedures

Table 2 reports final accuracy for every setting, and Table 9 the within-seed gains over plain. each setting changes the optimizer, activation, normalization, architecture, dataset or data size compared to reference. The transfer studies use three seeds each. The spatial ratios and schedule structure are preserved, with adaptations for map size and training budget. In particular, STL-10 (Coates et al., 2011) uses 96×96 inputs, $48 \rightarrow 72 \rightarrow 96$ resolutionreduction, Gaussian scales multiplied by three, and 60 epochs. SVHN (Netzer et al., 2011) uses a fixed 50,000-image training subset.

Table 2: Final test accuracy (%), mean $\pm$ sample SD over three seeds. **Bold means exceed Plain**; this is a descriptive comparison, not a significance test. All runs use the final epoch, without augmentation. BN: BatchNorm; GN: GroupNorm (8 groups). R, G and RG are defined in Table 1.

Model / normalization / activation	Optimizer	Epochs	Plain	R	G	RG
<i>CIFAR-10: 50 000 training / 10 000 test images</i>						
ResNet-20 / BN / ReLU	SGD	30	75.43 $\pm$ 0.76	80.37 $\pm$ 0.61	79.91 $\pm$ 0.27	81.51 $\pm$ 0.22
ResNet-20 / BN / ReLU	Adam	30	84.13 $\pm$ 0.59	85.31 $\pm$ 0.27	83.95 $\pm$ 0.07	85.28 $\pm$ 0.26
ResNet-20 / BN / ReLU	AdamW	30	84.12 $\pm$ 0.49	86.28 $\pm$ 0.22	83.71 $\pm$ 0.29	86.21 $\pm$ 0.29
ResNet-20 / BN / ReLU	RAdam	30	84.38 $\pm$ 0.12	85.63 $\pm$ 0.40	84.75 $\pm$ 0.19	85.26 $\pm$ 0.15
ResNet-20 / BN / GELU	SGD	30	74.80 $\pm$ 0.59	80.79 $\pm$ 0.42	80.27 $\pm$ 0.36	81.58 $\pm$ 0.13
ResNet-20 / BN / SiLU	SGD	30	76.55 $\pm$ 0.68	80.91 $\pm$ 0.33	80.44 $\pm$ 0.78	81.68 $\pm$ 0.14
ResNet-20 / GN / ReLU	SGD	30	75.48 $\pm$ 0.88	78.27 $\pm$ 0.68	76.64 $\pm$ 0.43	78.85 $\pm$ 0.21
VGG-11 / BN / ReLU	SGD	30	80.67 $\pm$ 0.61	83.77 $\pm$ 0.41	84.66 $\pm$ 0.25	84.54 $\pm$ 0.45
<i>SVHN: 50 000 training / 26 032 test images</i>						
ResNet-20 / BN / ReLU	SGD	30	93.10 $\pm$ 0.03	94.26 $\pm$ 0.07	94.26 $\pm$ 0.14	94.85 $\pm$ 0.20
<i>STL-10: 5 000 training / 8 000 test images</i>						
ResNet-20 / BN / ReLU	SGD	60	57.35 $\pm$ 0.53	59.93 $\pm$ 0.45	58.44 $\pm$ 1.55	56.06 $\pm$ 0.44
<i>CIFAR-10: 5 000 training / 10 000 test images</i>						
ResNet-20 / BN / ReLU	SGD	60	48.76 $\pm$ 1.05	55.04 $\pm$ 0.51	57.40 $\pm$ 0.38	59.39 $\pm$ 1.34
ResNet-20 / BN / ReLU	SGD	293	50.77 $\pm$ 0.93	55.00 $\pm$ 0.79	60.88 $\pm$ 0.24	62.55 $\pm$ 0.87

The first SGD row reuses the selection batch. Adaptive learning rates were selected using seed-0 test accuracy. Both 5k rows use the same training subset; 60 and 293 epochs correspond to 2 400 and 11 720 updates. Protocol details and paired gains are in Appendix C.

Two reduced-data settings use the same 5,000 CIFAR-10 training examples with either 60 or 293 epochss. The latter approximately matches the reference’s 293 epochs but does not match the number of distinct training examples. The training protocols, their adaptations to each setting, and the detailed results are given in Appendix C.

R improves mean accuracy in all 12 settings of the table, with positive paired differences in all 36 of the 36 seed pairs (Table 9). The mean gains range from 1.16 to 6.28 points. This consistency extends beyond the original activation and normalization: it holds with GELU, SiLU, GroupNorm and VGG-11-BN (Hendrycks and Gimpel, 2016; Ramachandran et al., 2017; Wu and He, 2018; Simonyan and Zisserman, 2015). This is evidence about these fixed recipes, rather than a guarantee for arbitrary architectures or training budgets.

The Gaussian gives a less uniform result. With Adam and AdamW (Kingma and Ba, 2015; Loshchilov and Hutter, 2019), G’s mean gain is slightly negative and varies in sign across seeds. With RAdam (Liu et al., 2020), it is positive but small. RG is close to R under Adam and AdamW, and below it under RAdam. The adaptive optimizers use learning rates selected on plain, seed 0, by final test accuracy; SGD retains the exploratory rate. Their baseline accuracies already exceed the SGD baseline by about nine points. These rows therefore compare interventions within optimizer

recipes, not equally tuned optimizers, and the reused test set limits their confirmatory interpretation.

The combined method does not perform best on every dataset. On STL-10, R and G improve the mean, but RG loses 1.29 points, with a negative difference in every seed. With 5,000 CIFAR-10 examples, RG instead gains 10.62 points at 2,400 updates and 11.78 at 11,720 updates. The longer budget does not remove this benefit. The opposing STL-10 and CIFAR-10-subset results show that sample count alone does not determine whether the combination helps; input resolution, scale adaptation and data distribution also differ.

4.3 Computational cost

We compare the time needed to obtain these accuracy gains. In the selection experiment, R takes around as long as Plain despite processing smaller feature maps, while G and RG increase total running time by approximately 23% and 45%, respectively (Table 3). We observe a similar pattern in the transfer experiments: R stays close to Plain, while Gaussian filtering usually adds cost. The measured times for all settings are reported in Appendix C.1.

These measurements include evaluation and checkpoint saving, and were collected on shared GPUs so timing differences possibly depend on the hardware and execu-

Table 3: Training cost in the selection batch: CIFAR-10 (50k), ResNet-20-BN/ReLU, SGD, 30 epochs on a Tesla T4. Time includes evaluation each epoch and checkpoint writes; mean $\pm$ SD over three seeds. Extra time is the mean within-seed percentage change from Plain.

Method	Time (min)	Extra time
Plain	8.42 ± 0.18	—
Resolution	8.32 ± 0.38	−1.1%
Gaussian	10.39 ± 0.11	+23.5%
Combined	12.22 ± 0.27	+45.2%

tion conditions. At inference however all added filters and resolution reductions are disabled, so computation is unchanged.

4.4 Training and test loss

In our initial experiments, we observed the interventions produced a higher final training CE, but lower test CE and higher test accuracy. This pattern also appears in many of the other settings. Across the nine settings with both training and test CE measurements, 26 method–baseline comparisons improve mean test accuracy. Of these, 22 show higher training-probe CE and lower test CE, while the remaining four show a decrease in both losses (Table 10). These comparisons use means over three seeds and a fixed 500-image training probe.

The separate five-seed reference study confirms the first pattern on the full training set: all three methods have higher training CE, lower test CE and higher test accuracy than Plain in every seed (Table 12). The gains therefore cannot be explained simply by a lower final training loss. They often occur despite a higher training loss, although this increase is not necessary for better test performance.

5 Loss geometry after spatial continuation

These differences between training and test performance motivate a closer look at the solutions reached by each procedure. We therefore examine the loss landscape around the trained models, using the same five-seed CIFAR-10 reference study discussed above and comparing Plain with R, G and RG. All final comparisons use the native, filter-free network. A fixed training probe and test probe each contain 1,000 class-balanced images. Larger checks use 10,000 training images and all 10,000 test images. Directions are repeated measurements of a checkpoint; the five training seeds, not the directions, are the replication units.

5.1 What is held fixed?

BatchNorm has learned affine parameters and running statistics. A loss surface depends on both. We evaluate three conventions: *saved*, which freezes the checkpoint statistics; *centre-frozen*, which recalibrates them once at the unperturbed weights and then freezes them; and *pointwise-recalibrated*, which recalibrates at every perturbed point. Calibration uses the same 2,000 training images, disjoint from the training probe, and no test data. The latter surface measures a different function: it includes the response of the statistics to the perturbation. It is not a correction that makes the other two surfaces invalid.

Following Li et al. (2018), we draw Gaussian directions and normalize each convolutional output filter, and each classifier row, to the norm of that weight block. With block index j ,

$$d_j = \|\theta_j\|_2 \frac{u_j}{\|u_j\|_2}, \quad u_j \sim \mathcal{N}(0, I). \quad (5)$$

Biases and BatchNorm affine parameters are held fixed. The same underlying draws are used across methods within a seed, then scaled by each model’s own norms. Thus we match *relative* perturbations, not identical displacements in parameter space.

For a specified data set and BatchNorm convention, write its evaluation loss as L . Our finite sensitivity is

$$S_\theta(\varepsilon) = \frac{1}{K} \sum_{k=1}^K \left[\frac{L(\theta + \varepsilon d_k) + L(\theta - \varepsilon d_k)}{2} - L(\theta) \right], \quad (6)$$

with $K = 20$ and ten amplitudes from 0.005 to 0.5. For each seed we form $\Delta S = S_{\text{method}} - S_{\text{plain}}$; a negative value means a smaller loss increase. The central loss is subtracted separately for every model and policy. S can be negative and is not clipped.

Figure 2 shows two-dimensional slices of this loss. Their appearance depends on the directions and normalization policy; the repeated-direction measurements in Figure 1 provide the quantitative comparison.

5.2 Finite perturbations depend on the evaluation policy

With *saved* or *centre-frozen* statistics, R and RG have lower S than plain in every seed, on both probes and at every tested amplitude. With *pointwise* recalibration, the training-probe difference remains negative for R and RG in every seed, but the test-probe effects are smaller and less uniform at the smallest amplitudes. The full-test check at $\varepsilon = 0.1$ gives mean differences -0.013 for R and -0.011 for RG, negative in 5 and 4 of the five seeds, respectively. At $\varepsilon = 0.25$ they are -0.084 and -0.093 , with the same counts (Table 13).

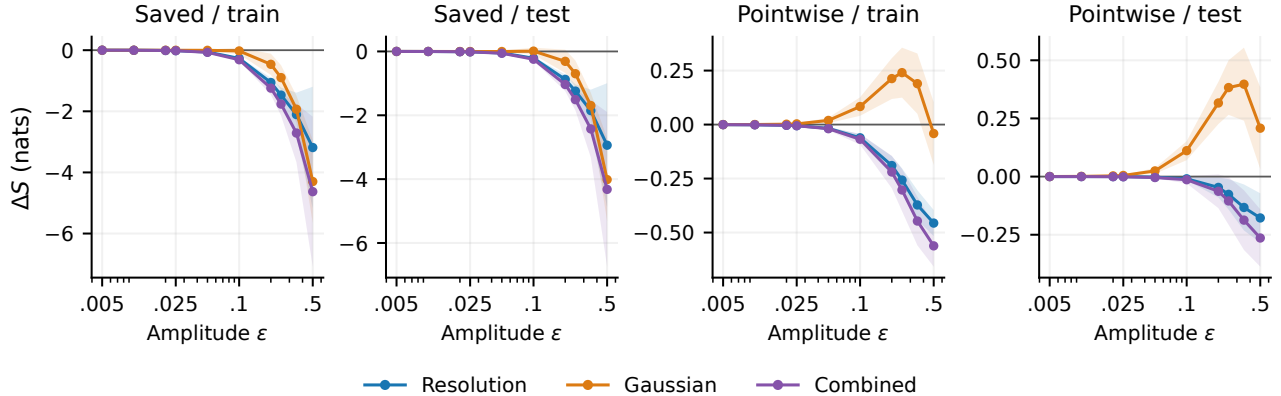


Figure 1: Finite sensitivity relative to Plain: CIFAR-10 / ResNet-20-BN / SGD, final 30-epoch checkpoints. Points average five seed-level differences on 1k-image probes; bands show ± 1 sample SD after averaging 20 directions per seed. Negative means a smaller CE increase. Amplitude axes are logarithmic; vertical scales differ between panels. Centre-frozen results and larger-set checks app ear in Appendix D.

Table 4: Leading Hessian eigenvalue relative to Plain, mean $\pm$ SD of five within-seed ratios. CIFAR-10 / ResNet-20-BN / SGD, 30 epochs; 1k-image probes, centre-frozen BatchNorm and block-relative coordinates. Plain is 1; absolute values appear in Table 15.

Method	Train probe	Test probe
Resolution	0.61 ± 0.07	0.51 ± 0.04
Gaussian	0.56 ± 0.09	0.47 ± 0.06
Combined	0.49 ± 0.04	0.40 ± 0.05

For G, recalibration changes the interpretation more substantially. With pointwise recalibration its test-probe sensitivity is higher than plain in every seed at every amplitude. Full-test differences are $+0.110$ at $\varepsilon = 0.1$ and $+0.379$ at 0.25 , positive in all five seeds. Nevertheless, G has better unperturbed test performance. The finite sensitivity measured by this policy therefore does not provide a common ordering that explains the accuracy gains of all three procedures.

5.3 Local Hessian measurements

A two-dimensional slice can miss directions of high or negative curvature. We therefore estimate the two largest algebraic eigenvalues and the smallest one of the CE Hessian, using automatic-differentiation Hessian-vector products and Lanczos iteration (Pearlmutter, 1994; Ghorbani et al., 2019). We use the same weight subset as in Equation (5), saved and centre-frozen statistics, and both ordinary and block-relative coordinates. The latter give $H_{\text{rel}} = A^T H A$, where $A_j = \|\theta_j\|_2 I$ is fixed at the checkpoint. We do not compute the Hessian of the pointwise-recalibrated function.

All three procedures have a smaller estimated largest eigenvalue than plain in every seed, under both coordinate choices, both fixed-statistics policies and both probes (Table 4). This includes G. The result is compatible with its larger finite sensitivity after pointwise recalibration because the quantities concern different functions and perturbation scales. Small negative eigenvalues are also found in all 160 eigenproblems. The gradients are nonzero, so these endpoints should not be called certified local minima. Appendix D.4 reports residuals, repeatability checks and the limits of finite-difference checks for the ReLU network.

5.4 Trajectories and input derivatives

Intermediate checkpoints, straight segments between final solutions and PCA projections are reported in Appendix D.5. They show path and state dependence, but neither disconnected basins nor a faithfully two-dimensional training trajectory. A separate exploratory grid examines derivatives of logits with respect to input pixels at fixed weights (Appendix E). The reference runs have lower mean input-Jacobian norms after each intervention, while their spatial-frequency summaries differ. This is complementary evidence; it does not turn the weight-space observations into a generalization theorem.

6 Discussion

The most consistent observation is practical: temporarily processing smaller internal feature maps improves the tested training recipes, and the added operation is absent from the final network. Gaussian filtering can provide a further gain, but that gain depends on

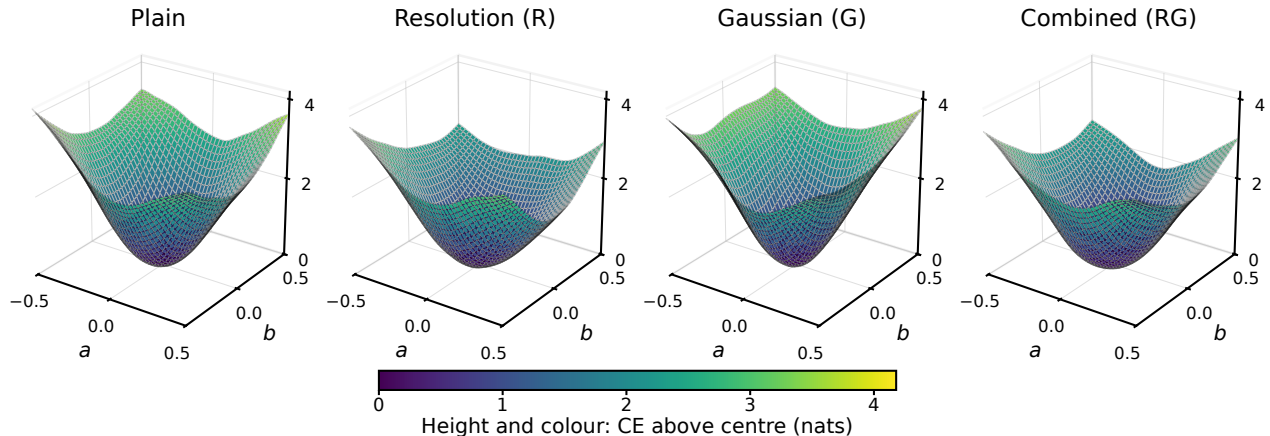


Figure 2: Illustrative loss slices for the four final seed-0 models: CIFAR-10 / ResNet-20-BN / SGD, 30 epochs, 1k-image training probe, pointwise BatchNorm recalibration. Surfaces join the 41-by-41 measured values of $L(\theta_0 + ad_1 + bd_2) - L(\theta_0)$. Viewing angles, axis limits and colour scales are identical. Directions share random draws but use each model’s filter norms. These slices illustrate relative perturbations in one seed, not a general ordering of curvature.

its placement, the optimizer and the dataset. Since the selected Gaussian and combined procedures differ in both placement and scale, the experiments do not identify separate additive effects of “resolution” and “blur.”

The geometric results also support a narrower conclusion than a single flatness explanation. Leading Hessian eigenvalues decrease under fixed statistics, while finite sensitivity depends on the BatchNorm convention and does not consistently show a smaller loss increase for Gaussian-only than for plain. Neither result establishes that curvature caused the improvement in test performance. Input derivatives, parameter derivatives and training–test gaps describe different aspects of a predictor and should not be treated as interchangeable measures.

The scope remains limited. The methods were selected through CIFAR-10 test performance; the adaptive learning rates also used that test set. Transfer covers a targeted collection of settings without augmentation, not a full factorial benchmark or uniformly tuned baselines. Three or five seeds quantify variation within these settings, not uncertainty over arbitrary tasks. Direct comparisons with CBS and SDPoint remain to be run. The present evidence motivates temporary internal resolution reduction as a simple training intervention and identifies conditions under which adding a Gaussian helps or hurts; it does not establish superiority over existing continuation methods or a theoretical guarantee.

References

- Allgower, E. L. and Georg, K. (1990). *Numerical Continuation Methods: An Introduction*, volume 13 of *Springer Series in Computational Mathematics*. Springer, Berlin, Heidelberg.
- Bengio, Y., Louradour, J., Collobert, R., and Weston, J. (2009). Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 41–48.
- Coates, A., Ng, A., and Lee, H. (2011). An analysis of single-layer networks in unsupervised feature learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, volume 15 of *Proceedings of Machine Learning Research*, pages 215–223.
- Dinh, L., Pascanu, R., Bengio, S., and Bengio, Y. (2017). Sharp minima can generalize for deep nets. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1019–1028.
- Garipov, T., Izmailov, P., Podoprikin, D., Vetrov, D., and Wilson, A. G. (2018). Loss surfaces, mode connectivity, and fast ensembling of DNNs. In *Advances in Neural Information Processing Systems*, volume 31.
- Ghorbani, B., Krishnan, S., and Xiao, Y. (2019). An investigation into neural net optimization via hessian eigenvalue density. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2232–2241.

- Gulcehre, C., Moczulski, M., Visin, F., and Bengio, Y. (2016). Mollifying networks. *arXiv preprint arXiv:1608.04980*.
- Hazan, E., Levy, K. Y., and Shalev-Shwartz, S. (2016). On graduated optimization for stochastic non-convex problems. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1833–1841.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778.
- Hendrycks, D. and Gimpel, K. (2016). Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*.
- Hoffer, E., Weinstein, B., Hubara, I., Ben-Nun, T., Hoefler, T., and Soudry, D. (2019). Mix & Match: Training convnets with mixed image sizes for improved accuracy, speed and scale resiliency. *arXiv preprint arXiv:1908.08986*.
- Hutchinson, M. F. (1989). A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics—Simulation and Computation*, 18(3):1059–1076.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pages 448–456.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations*.
- Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Technical report, University of Toronto.
- Kuen, J., Kong, X., Lin, Z., Wang, G., Yin, J., See, S., and Tan, Y.-P. (2018). Stochastic downsampling for cost-adjustable inference and improved regularization in convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- Li, H., Xu, Z., Taylor, G., Studer, C., and Goldstein, T. (2018). Visualizing the loss landscape of neural nets. In *Advances in Neural Information Processing Systems*, volume 31.
- Liu, L., Jiang, H., He, P., Chen, W., Liu, X., Gao, J., and Han, J. (2020). On the variance of the adaptive learning rate and beyond. In *International Conference on Learning Representations*.
- Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations*.
- Michaeli, H., Michaeli, T., and Soudry, D. (2023). Alias-free convnets: Fractional shift invariance via polynomial activations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16333–16342.
- Mobahi, H. and Fisher, III, J. (2015). A theoretical analysis of optimization by Gaussian continuation. In *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*.
- Netzer, Y., Wang, T., Coates, A., Bissacco, A., Wu, B., and Ng, A. Y. (2011). Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*.
- Pearlmutter, B. A. (1994). Fast exact multiplication by the hessian. *Neural Computation*, 6(1):147–160.
- Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., and Courville, A. (2019). On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 5301–5310.
- Ramachandran, P., Zoph, B., and Le, Q. V. (2017). Searching for activation functions. *arXiv preprint arXiv:1710.05941*.
- Simonyan, K. and Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*.
- Sinha, S., Garg, A., and Larochelle, H. (2020). Curriculum by smoothing. In *Advances in Neural Information Processing Systems*, volume 33, pages 21653–21664.
- Sokolić, J., Giryes, R., Sapiro, G., and Rodrigues, M. R. D. (2017). Robust large margin deep neural networks. *IEEE Transactions on Signal Processing*.
- Stergiou, A., Poppe, R., and Kalliatakis, G. (2021). Refining activation downsampling with SoftPool. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*.
- Tan, M. and Le, Q. (2021). EfficientNetV2: Smaller models and faster training. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10096–10106.
- Vasconcelos, C., Larochelle, H., Dumoulin, V., Romijnders, R., Le Roux, N., and Goroshin, R. (2021). Impact of aliasing on generalization in deep convolutional networks. In *Proceedings of the IEEE/CVF*

International Conference on Computer Vision, pages 10529–10538.

Wu, Y. and He, K. (2018). Group normalization. In *Proceedings of the European Conference on Computer Vision*.

Zhang, R. (2019). Making convolutional networks shift-invariant again. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 7324–7334.

A Reference protocol and method definitions

The comparisons that support our final selection use the protocol below. The three retained methods and the plain control are defined as they were executed in the final batch. Insertion sites, schedules and the final bypass are specified below.

A.1 Training recipe

Data	CIFAR-10, all 50,000 training and 10,000 test images; no augmentation; per-channel normalization with the statistics of the training images.
Model	ResNet-20 with BatchNorm, widths 16/32/64, option-A shortcuts (subsampling and zero-padding), 269,722 parameters; pinned initial weights per seed.
Optimizer	SGD, learning rate $\eta = 0.005$, momentum 0.9, weight decay 5×10^{-4} on all parameters; $\eta(u+1)/60$ for the first 60 updates, then cosine decay to zero over the global update index u .
Batches	A pinned permutation per epoch; batches of 128 images (the last one of 80), each split into microbatches of 32 whose losses are weighted by their number of examples. BatchNorm therefore normalizes over microbatches.
Budget	30 epochs of 391 updates, 11,730 updates in total.
Evaluation	Before training and after every epoch, on the test set and on a fixed probe of 500 training images, in eval mode with BatchNorm running statistics.

A.2 Conventions

Statistics and pairing. We report the record after epoch 30, without best-epoch selection, as the mean and sample standard deviation (SD) over seeds; a configuration with a single valid seed has no SD. Within a batch, runs with the same seed share their initial weights, BatchNorm buffers and data order, and we compute gains as means of per-seed paired differences. We do not pair runs across batches: plain runs on identical pinned assets finished up to 0.73 points apart in different batches, for reasons we have not established. Bars and bands in the figures show ± 1 sample SD; they are not confidence intervals.

Evaluation paths. The *current path* evaluates the network with the intervention of the epoch just completed. The *target path* evaluates the same weights at the native resolution without added filters, which is the network kept at the end. After epoch 30 the two paths coincide for every retained method. Before that, the target path uses BatchNorm running statistics accumulated on the current path, so it combines the effect of removing the intervention with a mismatch of these statistics.

Training loss. In the selection batch, training cross-entropy is evaluated on a fixed probe of 500 training images. It is not the full-training-set CE. Full-training-set CE was evaluated later for the twenty models used in the landscape study and for the separate 28-run diagnostic grid. The online mean loss during an epoch is another quantity: it is computed while the weights are changing.

Supplementary material. The complete exploration catalogue retains all 165 configurations, 377 runs and 19 experiments, including failures and separate launches. The accompanying material also contains per-run results, configurations and figure-generation scripts. [À compléter y faut ajouter la référence au matériel supp ou repo de code](#)

A.3 Insertion sites and the reduction point

“After block 1” denotes the complete output of the second residual block, after its residual addition and final ReLU. The adaptive max-pooling is applied to this tensor as it enters the third block, so both branches of every later block, including the shortcut, see the reduced grid. For an input side n and output side r , the one-dimensional adaptive pooling window is $[\lfloor in/r \rfloor, \lceil (i+1)n/r \rceil]$; the two-dimensional window is its Cartesian product across axes. Table 5 lists the sites in forward order.

In the combined method, each Gaussian sits between a convolution and its BatchNorm, so BatchNorm normalizes filtered activations. In the Gaussian-only method, the filtered tensor is the block output that the next block, or the pooling layer for p_9 , receives on both paths; the ReLU inside each block is not a site.

Table 5: Insertion sites of the ResNet-20 reference model. Each residual block has two convolutions, and stride-2 convolutions open blocks 3 and 6. Map sides are given for $r = 16/24/32$ at the reduction point; the Gaussian-only method never reduces, so its maps are those of the $r = 32$ column. Post-ReLU sites are the stem ReLU output (p_0) and the output of each block (p_1 – p_9).

Stage	Map side, $r = 16/24/32$	Conv outputs (COMBINED)	Post-ReLU (GAUSSIAN)
Stem	32 / 32 / 32	c_0	p_0
Blocks 0–1	32 / 32 / 32	c_1 – c_4	p_1, p_2
<i>Reduction (input of block 2)</i>	16 / 24 / 32	—	—
Block 2	16 / 24 / 32	c_5, c_6	p_3
Blocks 3–5	8 / 12 / 16	c_7 – c_{12}	p_4 – p_6
Blocks 6–8	4 / 6 / 8	c_{13} – c_{18}	p_7 – p_9

A.4 Schedules, operators and bypass

Table 6: Executed schedules, evaluated through the controller of the frozen presets. Epochs are zero-based; a state holds for a whole epoch of 391 updates. r is the side of the map entering residual block index 2 (RESOLUTION and COMBINED), G the reference Gaussian level. Effective σ is in pixels of the feature map at the site and is the same at every site of a method.

Epochs	First update	r	G	GAUSSIAN: σ (10 sites)	COMBINED: $\sigma = \frac{r}{32}G$ (19 sites)
0–2	0	16	1.00	1.000	0.500
3–5	1173	16	0.85	0.850	0.425
6–8	2346	24	0.70	0.700	0.525
9–11	3519	24	0.60	0.600	0.450
12–14	4692	32 (bypass)	0.50	0.500	0.500
15–17	5865	32 (bypass)	0.40	0.400	0.400
18–20	7038	32 (bypass)	0.30	0.300	0.300
21–29	8211	32 (bypass)	0.00	0 (bypass)	0 (bypass)

A state holds for a whole epoch: a transition at epoch e means that update $391e$ is the first to use the new state, and the record written after k completed epochs evaluates the current path under the state of epoch $k - 1$.

The Gaussian kernel has the same nine taps for every σ , $w_d \propto \exp(-d^2/(2\sigma^2))$ for $d = -4, \dots, 4$, normalized to sum one. It is applied separably and channel by channel, with reflection padding that does not repeat the edge sample; on 4×4 maps, where the radius equals the map side, the same reflection is applied by explicit indexing. The scale σ is measured in pixels of the feature map at the site.

The combined method uses $\sigma = (r/32)G$ at all 19 sites, including c_0 – c_4 , which remain at 32×32 upstream of the reduction. This rescaling comes from our earlier input-resolution arms (Appendix B) and makes the effective scale non-monotone: it increases from 0.425 to 0.525 when r changes from 16 to 24 at epoch 6. We keep this executed rule rather than a local width ratio, so the combined method is not the literal composition of the two other methods.

When $G = 0$, a Gaussian hook returns its input unchanged, and when $r = 32$ the reduction hook returns its input. In the final state ($r = 32, G = 0$), the augmented network therefore computes exactly the plain ResNet-20 when given the same weights and BatchNorm buffers. The trained predictors themselves need not coincide. Neither BatchNorm statistics nor optimizer state are reset at a transition.

B CIFAR-10 exploration and method selection

We used CIFAR-10 to explore interventions, to understand the first results and to select three methods for further study. This was not one homogeneous benchmark. Early pilots used image subsets, a validation split, GroupNorm or ResNet-18, and different budgets. Their numbers cannot be placed on the same ranking as the later reference experiments. The final selection rests on one paired batch under Appendix A. The same test set informed successive decisions, so the resulting scores are exploratory evidence, not an independent validation of the selected methods. The complete catalogue is retained in the supplementary material.

B.1 From input blur to feature-map filtering

We first filtered the input images, using ResNet-20 with GroupNorm, 45,000 training and 5,000 validation images, and 14,040 updates. Fixed blur lowered validation accuracy: 82.55% for $\sigma = 0.5$ and 71.33% for $\sigma = 1$, against 84.11% without blur (three seeds). Warm starts that blurred the images for the first 1,500 updates before removing the blur, in one or two steps, did not recover the unfiltered result (82.93% and 83.19%).

We then moved the Gaussian to the 19 convolution outputs, following the feature-map filtering idea of Sinha et al. (2020). On a 10,000-image subset with GroupNorm, it lowered validation accuracy ($46.91 \pm 0.60\%$ against $50.36 \pm 0.52\%$, three seeds). Two one-seed pilots with BatchNorm gave the opposite sign: 62.16% against 54.26% for ResNet-18, and 59.62% against 55.36% for ResNet-20. Several conditions changed, so these pilots do not isolate a normalization effect. Under the reference protocol, the Gaussian reached $78.41 \pm 0.32\%$ with plateaus and $78.46 \pm 0.72\%$ with geometric decay, against $75.40 \pm 0.40\%$ for plain. We continued with the reference ResNet-20-BN and plateau schedule.

B.2 Processing smaller representations

In a one-seed reference pilot, antialiased bilinear resizing of the input to $16 \rightarrow 24 \rightarrow 32$ reached 79.57%, against 75.10% for plain. Adding convolution-output filtering, with scale multiplied by $r/32$, gave 79.93%. A grid of 21 configurations with three seeds varied input versus stem reduction, bilinear versus max-pooling, schedules, and Gaussian combinations. Input reduction with the Gaussian reached $80.71 \pm 0.65\%$, compared with $78.69 \pm 0.68\%$ for the Gaussian alone and $75.62 \pm 0.49\%$ for plain. This led us to study resolution reduction as an intervention in its own right.

B.3 Operators, insertion points and schedules

An ablation batch suggested reducing after an early residual block. The subsequent resolution-only study compared 29 targeted configurations with three seeds: seven operators at the input and after block 1, max-pooling at five locations, four schedules for selected operators, and fixed-resolution controls. This was not a full factorial design. Of 87 runs, 84 were numerically valid; the perceptual operator after block 1 produced non-finite losses in all three seeds.

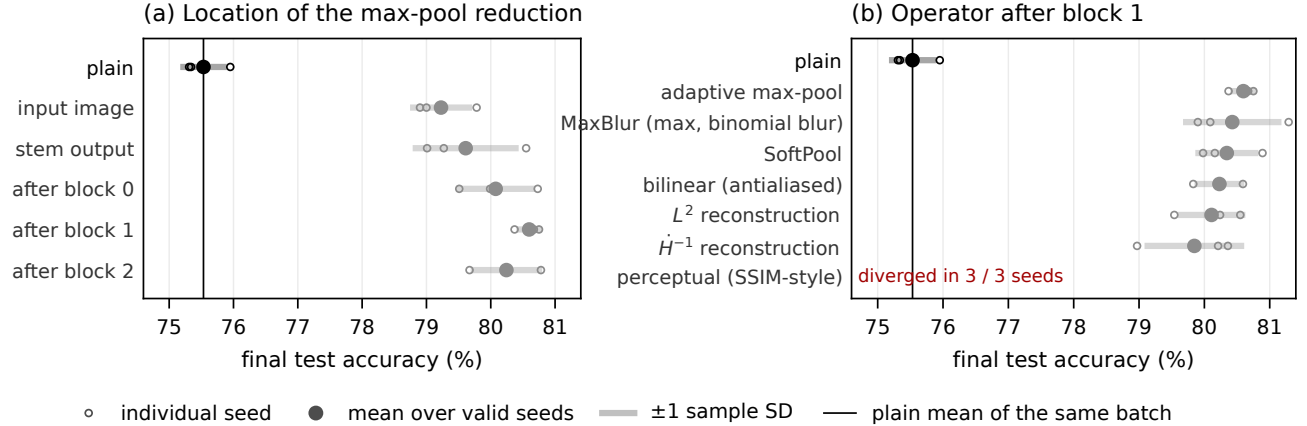


Figure 3: Resolution-only exploration with the $16 \rightarrow 24 \rightarrow 32$ schedule, three seeds per arm. Left: max-pooling insertion point. Right: reduction operator after block 1. The vertical line is this batch’s plain mean; bars show sample SD. These scores guided selection on CIFAR-10.

Max-pooling after block 1 had the highest mean among its locations ($80.60 \pm 0.20\%$). The six valid operators at that point had overlapping seed ranges and none exceeded its mean (Table 7). At the input, the perceptual and homogeneous $\dot{H}^{-1}$ reconstructions had higher means than max-pooling. We do not interpret these small differences as a definitive operator ranking.

For max-pooling, the progressive $16 \rightarrow 24 \rightarrow 32$ schedule scored above the tested delayed, shorter and reversed

Table 7: Resolution-only batch (no additional Gaussian continuation): reduction operator at two locations, schedule $16 \rightarrow 24 \rightarrow 32$. Final test accuracy after 30 epochs, mean $\pm$ sample SD over three seeds. Plain arm of the batch: $75.53 \pm 0.36\%$. The perceptual operator after block 1 produced a non-finite loss in all three seeds.

Reduction operator	After block 1 (%)	At the input (%)
Adaptive max-pool	80.60 ± 0.20	79.23 ± 0.48
MaxBlur (max, binomial blur)	80.43 ± 0.75	79.17 ± 0.43
SoftPool	80.34 ± 0.48	79.28 ± 0.46
Bilinear (antialiased)	80.23 ± 0.38	79.56 ± 0.68
L^2 reconstruction	80.11 ± 0.52	79.64 ± 0.33
$\hat{H}^{-1}$ reconstruction	79.85 ± 0.76	79.76 ± 0.65
Perceptual (SSIM-style)	Diverged (0/3 valid)	79.98 ± 0.40

schedules. MaxBlur behaved differently: the reversed order gave 80.46%, against 80.43% for the progressive order. Keeping the reduced grid to the end gave 76.15% at 24 and 73.13% at 16. These controls retain a different inference computation, so they do not isolate restoration of resolution at equal inference cost. We retained adaptive max-pooling after block 1 as a simple, competitive operator.

B.4 Final paired comparison

The last selection batch contained 16 configurations and three seeds with matched assets. It compared plain, Gaussian placement, reduction locations and combinations with different per-site scale profiles. Without

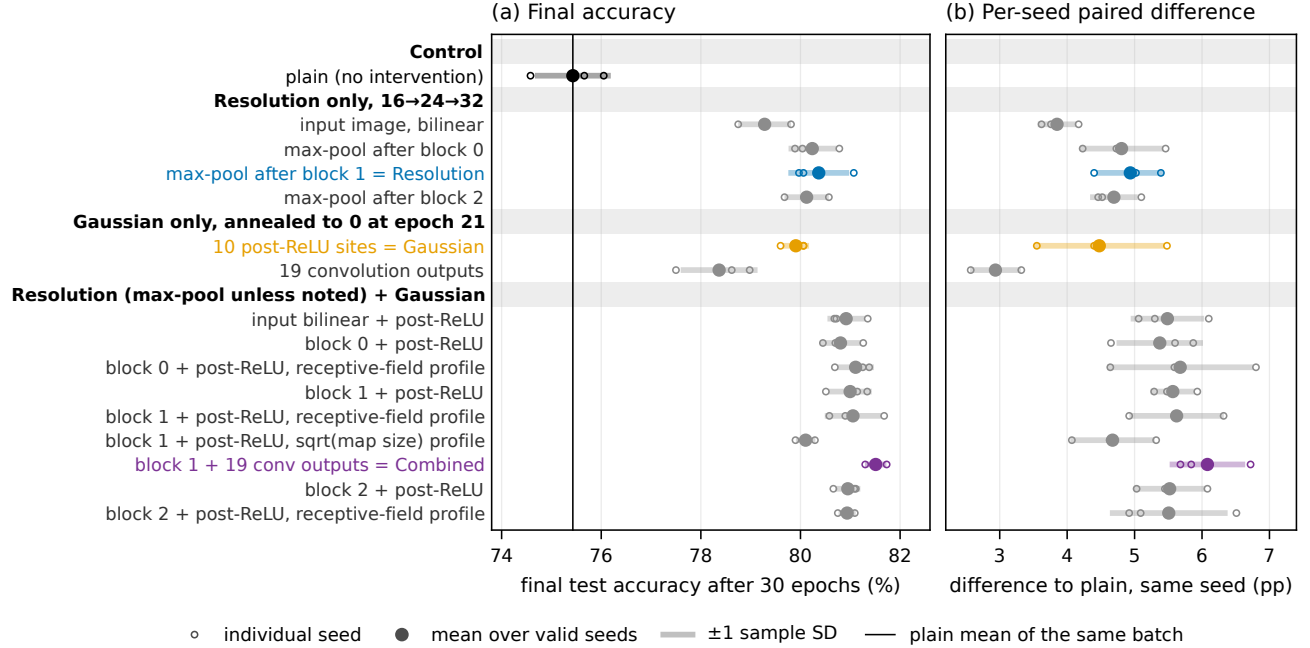


Figure 4: All 16 configurations of the final selection batch. Left: final test accuracy. Right: difference to plain within the same seed. Colours identify the retained procedures. This is one internally paired batch; it does not remove the reuse of CIFAR-10 test performance during selection.

reduction, post-ReLU filtering scored above convolution-output filtering (79.91% versus 78.37%; paired difference 1.54 ± 0.88 points). With reduction after block 1, the order reversed: convolution-output filtering reached 81.51% against 81.00% for post-ReLU filtering, a paired difference of 0.51 ± 0.24 points. Both differences were positive in all three seeds. Per-site scale profiles did not give a consistent further benefit; six other combined arms finished within 0.6 points of the highest mean.

We selected one representative per family: max-pooling after block 1; the uniform Gaussian at ten post-ReLU

Table 8: Endpoints of the final selection batch: CIFAR-10 / ResNet-20-BN / SGD, 30 epochs. Mean $\pm$ sample SD over three seeds; gains are paired to Plain. Test and 500-image training-probe CE use saved BatchNorm statistics. Last-epoch loss is the online mean while weights are updated. All losses are in nats; full-training-set CE was not evaluated in this batch. Bold accuracy means exceed Plain.

Method	Test acc. (%)	Gain (pp)	Test CE	Probe CE	Last-epoch loss
Plain	75.43 $\pm$ 0.76	—	0.762 $\pm$ 0.012	0.211 $\pm$ 0.024	0.274 $\pm$ 0.022
Resolution	80.37 $\pm$ 0.61	+4.94 $\pm$ 0.50	0.578 $\pm$ 0.011	0.321 $\pm$ 0.011	0.358 $\pm$ 0.008
Gaussian	79.91 $\pm$ 0.27	+4.48 $\pm$ 0.97	0.593 $\pm$ 0.008	0.281 $\pm$ 0.006	0.339 $\pm$ 0.002
Combined	81.51 $\pm$ 0.22	+6.08 $\pm$ 0.56	0.542 $\pm$ 0.006	0.318 $\pm$ 0.005	0.369 $\pm$ 0.002

sites; and max-pooling after block 1 with filtering at 19 convolution outputs. The uniform post-ReLU method was the better of the two basic Gaussian placements in this paired batch. It was not the highest mean among every historical Gaussian profile, and the selected procedures should not be called established family optima.

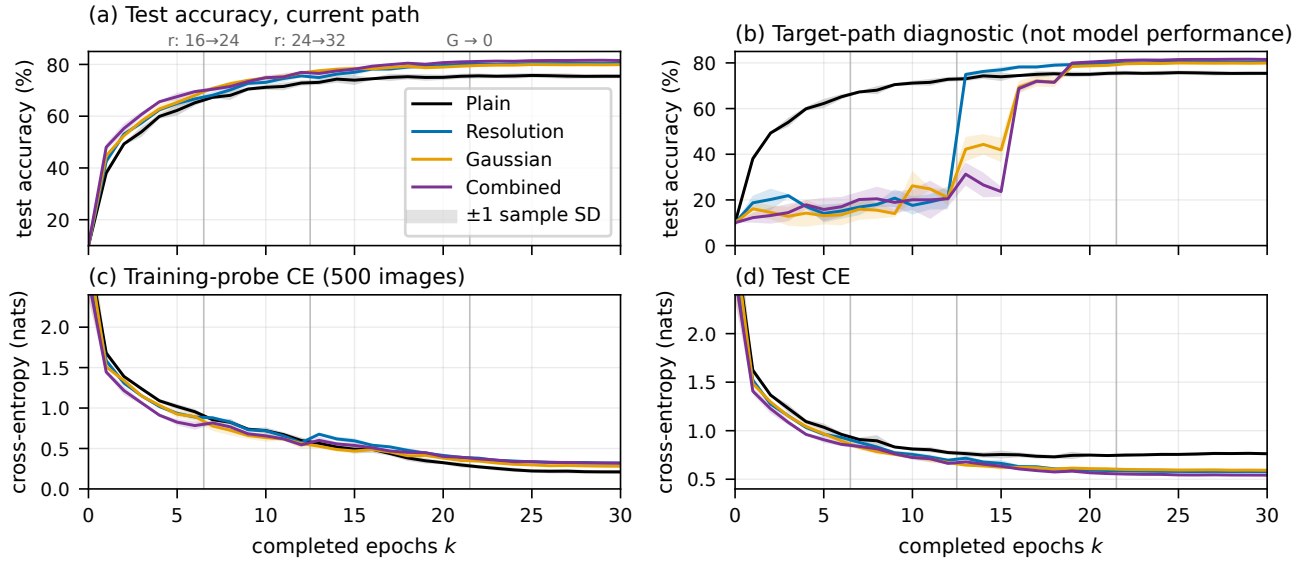


Figure 5: Selection-batch training records, means over three seeds. The current path retains the intervention of the epoch just completed; the target path removes it using saved BatchNorm statistics. Before restoration, the target diagnostic mixes the changed computation with a normalization-statistics mismatch. Training-probe CE uses 500 fixed images.

B.5 Other explored operators

SoftPool uses exponentially weighted local activations (Stergiou et al., 2021); MaxBlur combines max-pooling with binomial smoothing. The reconstruction operators address a different question: which low-resolution coefficients best reconstruct a channel h through bilinear upsampling U_r ? They minimize

$$\frac{1}{2}(U_r z - h)^\top Q(U_r z - h), \quad \mathbf{1}^\top(U_r z - h) = 0,$$

with $Q = I$ or the pseudoinverse of a spatial Laplacian. Unlike a TV constraint, this is a quadratic reconstruction objective on a smaller coefficient grid. The perceptual variant uses an SSIM-style reconstruction criterion; its divergence inside the network is retained as a failed experiment, not removed from the catalogue.

A one-seed db2 wavelet pilot on a 10,000-image subset reached 56.34% validation accuracy against 55.40% for plain, at 3856s versus 89s per run. Adaptive schedules and alternative allocations of time between blur levels did not clearly outperform fixed-schedule reruns in their one-seed study. Finally, longer one-seed Gaussian runs illustrate budget dependence: at 120 epochs, the convolution-output plateau schedule gave 80.68% against 77.10%

at the reference learning rate, and 87.16% against 86.04% at ten times that rate. These exploratory controls are not equally tuned comparisons of the final three methods.

C Transfer protocols and complete endpoint metrics

The transfer experiments were run after selecting the three procedures. The matrix is targeted: changing several axes does not amount to crossing every dataset with every architecture and optimizer. All methods within a setting use the same training budget, examples, per-seed initial state and ordering. New architectures require new initial weights, so their seeds are not paired with weights of a different architecture. No study uses augmentation. Final test accuracy is always taken at the scheduled endpoint.

Optimizer and architecture changes on CIFAR-10. All full-data CIFAR-10 settings use 50,000 training images, all 10,000 test images and 30 epochs of 391 updates. Batch size is 128, accumulated over microbatches of 32; warm-up, cosine decay and numerical weight decay 5×10^{-4} follow Appendix A. The ResNet-20-BN/ReLU optimizer comparison uses learning rates 0.005, 0.003, 0.02 and 0.003 for SGD, Adam, AdamW and RAdam, respectively. All activation, normalization and architecture variants use SGD at 0.005.

GELU and SiLU replace the functional activations but retain the pinned initial weights and their Kaiming-ReLU gain. GroupNorm uses eight groups, new initial weights and the reference data order; its models have no running-statistics policy. VGG-11-BN has eight convolution-output and eight post-ReLU sites. Reduction follows its first pooling layer, with map side $8 \rightarrow 12 \rightarrow 16$; four later native pools can bring the last maps to 1×1 . Its weights are new, while data indices and order match the reference.

Dataset and budget changes with ResNet-20-BN/ReLU and SGD. SVHN uses 50,000 of its 73,257 training images and all 26,032 test images, with learning rate 0.005, 30 epochs, 11,730 updates and the reference 32-pixel scales. STL-10 uses all 5,000 labelled training and 8,000 test images at 96×96 , the same learning rate and 60 epochs of 40 updates. Reduction becomes $48 \rightarrow 72 \rightarrow 96$, Gaussian scales are multiplied by three (25 taps per axis), and epoch boundaries are doubled. Evaluation occurs every third epoch and at the endpoint.

The two CIFAR-10 5k settings use the same 5,000 training images and all 10,000 test images, with the same model and SGD rate. The 60-epoch setting follows STL-10’s 2,400-update budget and evaluation frequency, with 32-pixel spatial scales. The 293-epoch setting uses 11,720 updates, approximately matching the full-data reference. Reference epoch boundaries are scaled by $391/40$ and rounded to whole epochs: transitions occur at updates 1,160, 2,360, 3,520, 4,680, 5,880, 7,040 and 8,200. Evaluation occurs every tenth epoch and at the endpoint.

Optimizer selection. The optimizer study contains twelve imported selection-batch runs, 36 runs at the selected adaptive learning rates, fourteen learning-rate pilot runs and three SGD consistency runs. Only the first two groups enter Table 2. Pilot choices used final plain seed-0 *test* accuracy, with a 0.3-point near-tie preference for the lower rate and extensions when a grid edge was competitive. The chosen rate was then shared by plain and all interventions; it was not tuned separately for each method. SGD was not retuned. The stronger adaptive baselines and the smaller intervention gains therefore concern different selected recipes, with test-set reuse. Equal numerical weight decay does not make SGD/Adam and AdamW equivalent regularization schemes (Loshchilov and Hutter, 2019).

Training CE is evaluated on a fixed 500-image probe. In both CIFAR-10 5k settings, the probe is contained in the same 5,000-image training subset shared by all methods and budgets. Per-channel normalization uses the full 50,000-image training split. These CEs describe a probe, not the full training set.

The training-probe and test CEs help delimit the initial training–test observation. For example, after 293 epochs on the 5,000-image CIFAR-10 subset, every method has 100% training-probe accuracy and very small CE, while test CE remains substantially lower under G and RG. This is compatible with a difference in the fitted predictors, but does not by itself identify a regularization mechanism. Conversely, R improves both training-probe and test CE with GroupNorm and on STL-10. A statement that continuation always sacrifices training fit would be false for these experiments.

Training CNNs with Spatial Continuation

Table 9: Paired accuracy gains over Plain (percentage points), mean $\pm$ sample SD over three seeds. Configurations and evaluation sets match Table 2; differences are formed within each setting and seed.

Model / normalization / activation	Optimizer	Epochs	Δ R	Δ G	Δ RG
<i>CIFAR-10: 50 000 training / 10 000 test images</i>					
ResNet-20 / BN / ReLU	SGD	30	$+4.94 \pm 0.50$	$+4.48 \pm 0.97$	$+6.08 \pm 0.56$
ResNet-20 / BN / ReLU	Adam	30	$+1.18 \pm 0.40$	-0.18 ± 0.61	$+1.15 \pm 0.36$
ResNet-20 / BN / ReLU	AdamW	30	$+2.16 \pm 0.68$	-0.41 ± 0.72	$+2.09 \pm 0.62$
ResNet-20 / BN / ReLU	RAdam	30	$+1.25 \pm 0.53$	$+0.37 \pm 0.18$	$+0.88 \pm 0.26$
ResNet-20 / BN / GELU	SGD	30	$+5.99 \pm 0.48$	$+5.48 \pm 0.85$	$+6.78 \pm 0.48$
ResNet-20 / BN / SiLU	SGD	30	$+4.36 \pm 0.96$	$+3.90 \pm 1.18$	$+5.13 \pm 0.72$
ResNet-20 / GN / ReLU	SGD	30	$+2.79 \pm 0.80$	$+1.16 \pm 0.63$	$+3.37 \pm 0.67$
VGG-11 / BN / ReLU	SGD	30	$+3.10 \pm 0.37$	$+3.99 \pm 0.63$	$+3.88 \pm 0.58$
<i>SVHN: 50 000 training / 26 032 test images</i>					
ResNet-20 / BN / ReLU	SGD	30	$+1.16 \pm 0.09$	$+1.16 \pm 0.12$	$+1.74 \pm 0.17$
<i>STL-10: 5 000 training / 8 000 test images</i>					
ResNet-20 / BN / ReLU	SGD	60	$+2.58 \pm 0.13$	$+1.09 \pm 1.07$	-1.29 ± 0.53
<i>CIFAR-10: 5 000 training / 10 000 test images</i>					
ResNet-20 / BN / ReLU	SGD	60	$+6.28 \pm 0.78$	$+8.64 \pm 1.22$	$+10.62 \pm 0.35$
ResNet-20 / BN / ReLU	SGD	293	$+4.23 \pm 0.21$	$+10.10 \pm 0.87$	$+11.78 \pm 0.06$

Gains are positive in all three seed pairs except G with Adam (2/3); G with AdamW (1/3); RG with STL-10 (0/3).

Table 10: Final cross-entropy (nats), mean $\pm$ sample SD over three seeds. All rows use SGD, learning rate 0.005. Train uses a fixed 500-image probe; Test uses the complete test set. Adaptive-optimizer endpoint CEs were not recorded.

Model / norm. / activation	Split	Plain	R	G	RG
<i>CIFAR-10: 50 000 training / 10 000 test images</i>					
ResNet-20 / BN / ReLU	Train	0.211 ± 0.024	0.321 ± 0.011	0.281 ± 0.006	0.318 ± 0.005
	Test	0.762 ± 0.012	0.578 ± 0.011	0.593 ± 0.008	0.542 ± 0.006
ResNet-20 / BN / GELU	Train	0.220 ± 0.047	0.307 ± 0.012	0.255 ± 0.021	0.327 ± 0.008
	Test	0.792 ± 0.030	0.571 ± 0.009	0.590 ± 0.010	0.539 ± 0.003
ResNet-20 / BN / SiLU	Train	0.255 ± 0.010	0.322 ± 0.027	0.294 ± 0.023	0.340 ± 0.006
	Test	0.728 ± 0.028	0.557 ± 0.004	0.581 ± 0.018	0.532 ± 0.005
ResNet-20 / GN / ReLU	Train	0.514 ± 0.025	0.479 ± 0.007	0.497 ± 0.009	0.472 ± 0.034
	Test	0.714 ± 0.022	0.635 ± 0.011	0.674 ± 0.012	0.614 ± 0.001
VGG-11 / BN / ReLU	Train	0.000157 ± 0.000024	0.000746 ± 0.000028	0.000743 ± 0.000130	0.00209 ± 0.00033
	Test	0.881 ± 0.032	0.668 ± 0.019	0.641 ± 0.012	0.593 ± 0.009
<i>SVHN: 50 000 training / 26 032 test images</i>					
ResNet-20 / BN / ReLU	Train	0.0336 ± 0.0014	0.105 ± 0.007	0.0696 ± 0.0078	0.107 ± 0.011
	Test	0.248 ± 0.002	0.202 ± 0.003	0.202 ± 0.002	0.183 ± 0.006
<i>STL-10: 5 000 training / 8 000 test images</i>					
ResNet-20 / BN / ReLU	Train	0.903 ± 0.019	0.832 ± 0.010	0.979 ± 0.035	1.10 ± 0.03
	Test	1.156 ± 0.014	1.084 ± 0.015	1.143 ± 0.032	1.212 ± 0.019
<i>CIFAR-10: 5 000 training / 10 000 test images</i>					
ResNet-20 / BN / ReLU	Train	0.218 ± 0.070	0.432 ± 0.040	0.362 ± 0.028	0.457 ± 0.018
	Test	1.688 ± 0.094	1.329 ± 0.016	1.263 ± 0.005	1.175 ± 0.045
ResNet-20 / BN / ReLU	Train	0.000589 ± 0.000036	0.000980 ± 0.000103	0.00222 ± 0.00018	0.00296 ± 0.00030
	Test	2.454 ± 0.059	2.081 ± 0.080	1.662 ± 0.016	1.538 ± 0.023

C.1 Observed computational cost

The timer starts after data and model construction and ends after the last evaluation. It includes evaluation and checkpoint writes. Subtracting evaluation time still leaves indexing and I/O; it would not isolate GPU

Table 11: Total wall time in minutes, mean $\pm$ sample SD over three seeds. Evaluation and checkpoint writes are included. The column k means evaluation every k epochs, plus the endpoint. Compare methods within a setting: GPU sessions and evaluation workloads differ across settings.

Model / normalization / activation	Optimizer	Epochs	k	Plain	R	G	RG
<i>CIFAR-10: 50 000 training / 10 000 test images</i>							
ResNet-20 / BN / ReLU	SGD	30	1	8.42 ± 0.18	8.32 ± 0.38	10.39 ± 0.11	12.22 ± 0.27
ResNet-20 / BN / ReLU	Adam	30	1	8.21 ± 0.11	8.27 ± 0.24	10.36 ± 0.09	12.27 ± 0.11
ResNet-20 / BN / ReLU	AdamW	30	1	8.29 ± 0.15	8.22 ± 0.28	10.31 ± 0.03	12.13 ± 0.04
ResNet-20 / BN / ReLU	RAdam	30	1	8.30 ± 0.27	8.21 ± 0.24	10.46 ± 0.10	12.34 ± 0.12
ResNet-20 / BN / GELU	SGD	30	1	7.23 ± 0.11	7.58 ± 0.07	10.09 ± 0.89	11.46 ± 0.32
ResNet-20 / BN / SiLU	SGD	30	1	7.78 ± 0.67	7.97 ± 0.48	9.85 ± 0.26	11.66 ± 0.45
ResNet-20 / GN / ReLU	SGD	30	1	7.96 ± 0.56	7.69 ± 0.09	10.30 ± 0.38	11.82 ± 0.30
VGG-11 / BN / ReLU	SGD	30	1	10.14 ± 0.71	9.83 ± 0.08	14.11 ± 0.55	13.17 ± 0.37
<i>SVHN: 50 000 training / 26 032 test images</i>							
ResNet-20 / BN / ReLU	SGD	30	1	8.16 ± 0.31	8.31 ± 0.14	11.17 ± 0.09	12.59 ± 0.16
<i>STL-10: 5 000 training / 8 000 test images</i>							
ResNet-20 / BN / ReLU	SGD	60	3	10.43 ± 0.44	10.39 ± 0.25	17.36 ± 0.90	18.61 ± 0.33
<i>CIFAR-10: 5 000 training / 10 000 test images</i>							
ResNet-20 / BN / ReLU	SGD	60	3	1.65 ± 0.02	1.75 ± 0.03	2.31 ± 0.07	2.72 ± 0.07
ResNet-20 / BN / ReLU	SGD	293	10	7.44 ± 0.18	7.46 ± 0.10	10.01 ± 0.62	11.70 ± 0.77

Table 12: Endpoints of the separate five-seed landscape batch: CIFAR-10 / ResNet-20-BN / SGD, 30 epochs, saved BatchNorm. Accuracy uses all 10k test images; CE uses all 50k training or 10k test images. Mean $\pm$ sample SD. Bold accuracy means exceed Plain; this batch is not pooled with Table 2.

Method	Test accuracy (%)	Train CE (nats)	Test CE (nats)
Plain	75.65 ± 0.78	0.197 ± 0.019	0.759 ± 0.010
Resolution	80.21 ± 0.36	0.303 ± 0.008	0.584 ± 0.010
Gaussian	80.12 ± 0.49	0.273 ± 0.006	0.587 ± 0.021
Combined	81.58 ± 0.30	0.312 ± 0.006	0.542 ± 0.005

computation. The reference runs use a Tesla T4, while hardware identifiers are not recorded in the transfer export. Runs used shared GPU sessions, sometimes with two jobs per device. Evaluation frequency and the number of evaluated paths also vary across studies. These timings therefore describe the observed cost of each recipe rather than a controlled hardware benchmark.

R stays close to Plain in elapsed time, while Gaussian filtering generally adds cost. Smaller early feature maps do not by themselves imply an end-to-end speedup. No energy or peak-memory measurements are available, and we do not infer them from wall time. At inference, all methods use the original architecture without added filtering or reduction.

D Weight-space diagnostics and numerical checks

The geometry study uses twenty separately trained reference models: plain, R, G and RG at seeds 0–4. Initial weights and data order are paired within each seed. All new amplitude, temporal and Hessian evaluations reuse those checkpoints; no additional optimization is needed for these diagnostics. The five-seed batch is separate from the three-seed selection batch, so its repeated seed labels do not justify pooling the runs.

D.1 Evaluation sets and normalization

The training and test probes each contain 1,000 fixed, class-balanced examples. Calibration uses a fixed 2,000-image training subset, disjoint from the training probe, in four batches of 500. Larger finite-sensitivity checks use 10,000 training images and the full test set; the larger training subset includes the training probe but is disjoint

from calibration. Calibration resets running statistics and accumulates their cumulative batch averages without gradient updates. Evaluation then uses these buffers in eval mode. No test examples calibrate BatchNorm, and previous grid points do not leave their buffers in the next evaluation.

Writing b for the BatchNorm buffers and $\hat{b}(\theta)$ for this calibration procedure clarifies the three functions measured:

$$L_{\text{saved}}(\theta) = L(\theta; b_{\text{checkpoint}}), \quad L_{\text{centre}}(\theta) = L(\theta; \hat{b}(\theta_0)), \quad L_{\text{point}}(\theta) = L(\theta; \hat{b}(\theta)).$$

The calibration batch size, order and procedure are part of the definition of $\hat{b}$; it is not an exact population statistic. In centre-frozen evaluations, θ_0 is the checkpoint about which the plot or derivative is computed. Saved and centre-frozen policies keep the buffers constant under weight perturbations. Pointwise recalibration includes their change. At intermediate checkpoints, the intervention state is also specified independently of the buffers.

D.2 Direction sampling, aggregation and amplitudes

The perturbed vector contains 268,336 convolution and classifier weights, arranged in 698 output-filter or classifier-row blocks. Biases and BatchNorm affine parameters are fixed. Every measured block has a nonzero norm. Gaussian draws are generated deterministically, shared across methods within a seed, and normalized as in Equation (5). One useful displacement scale is

$$r(\delta) = \left(\frac{1}{698} \sum_j \frac{\|\delta_j\|_2^2}{\|\theta_j\|_2^2} \right)^{1/2}.$$

For the random directions, $r(\varepsilon d) = |\varepsilon|$. Directions normalized using different checkpoints need not have the same physical length or span the same plane.

The amplitudes are 0.005, 0.01, 0.02, 0.025, 0.05, 0.1, 0.2, 0.25, 0.35, 0.5. Twenty directions per model give $20 \times 20 \times 3 \times 2 \times 10 = 24,000$ directional sensitivity values across networks, policies, probes and amplitudes. We first average over directions within a checkpoint, then pair each method with plain in that seed, and only then average across the five seeds. Repeated directions and amplitudes do not increase the number of independent training replications. Sample SDs over seeds are descriptive spreads, not confidence intervals or hypothesis tests.

For a twice differentiable fixed function and a sufficiently small displacement,

$$\frac{2S_\theta(\varepsilon)}{\varepsilon^2} \rightarrow \frac{1}{K} \sum_k d_k^\top H d_k.$$

The symmetric evaluation cancels the first-order term. At finite amplitudes, ReLU activation changes and nonquadratic terms can invalidate that approximation; pointwise recalibration also changes the function. We therefore call S a finite sensitivity, not a Hessian eigenvalue or a universal flatness measure.

D.3 What the surfaces show

A random plane evaluates $L(\theta_0 + ad_1 + bd_2)$ on a grid. The axes are two chosen directions in parameter space; they are not coordinates of input images. An absolute surface displays the CE itself. A centred surface displays $L(\theta_0 + ad_1 + bd_2) - L(\theta_0)$, so every centre has height zero even though the quoted centre CEs differ. This removes the vertical offset, not the shape. The smooth bowl-like appearance of a sampled plane does not establish convexity in all parameter directions. Figure 2 in the main text shows the final seed-0 planes with pointwise recalibration, using the same viewing angle, axis limits and colour scale for every method. The surfaces join all 41-by-41 measured vertices; no additional training seeds are represented by the plane.

D.4 Hessian definitions and convergence

The Hessian is that of mean CE, excluding weight decay, on the selected weight subset at the final checkpoint. The ordinary operator is $H = \nabla_\theta^2 L$. For block-relative coordinates, $\theta = \theta_0 + Aq$ with $A_j = \|\theta_{0,j}\|_2 I$, we use $H_{\text{rel}} = A^\top H A$ and keep A fixed during differentiation. Normalizing by filter norms reduces an obvious scale mismatch but does not give invariance to arbitrary reparameterizations (Dinh et al., 2017). It also does not make

Table 13: Finite sensitivity S (nats) on larger evaluation sets. CIFAR-10, ResNet-20-BN/ReLU, SGD, final 30-epoch checkpoints of the five-seed landscape batch. Entries are means $\pm$ sample SD across seeds, each averaging 20 directions. S is the centre-subtracted CE increase in Equation (6), not the unperturbed loss.

Evaluation set	ϵ	Plain	R	G	RG
<i>Saved BatchNorm statistics</i>					
Train subset (10k)	0.1	0.732 ± 0.060	0.468 ± 0.028	0.726 ± 0.086	0.431 ± 0.044
Train subset (10k)	0.25	5.269 ± 0.356	3.815 ± 0.294	4.423 ± 0.184	3.528 ± 0.319
Full test set (10k)	0.1	0.625 ± 0.054	0.407 ± 0.027	0.633 ± 0.082	0.386 ± 0.039
Full test set (10k)	0.25	4.866 ± 0.360	3.616 ± 0.283	4.170 ± 0.199	3.370 ± 0.312
<i>BatchNorm recalibrated at each point</i>					
Train subset (10k)	0.1	0.192 ± 0.015	0.135 ± 0.005	0.283 ± 0.048	0.128 ± 0.025
Train subset (10k)	0.25	1.020 ± 0.067	0.769 ± 0.022	1.283 ± 0.120	0.729 ± 0.098
Full test set (10k)	0.1	0.102 ± 0.009	0.089 ± 0.004	0.213 ± 0.043	0.092 ± 0.023
Full test set (10k)	0.25	0.700 ± 0.061	0.616 ± 0.022	1.079 ± 0.119	0.607 ± 0.096

Table 14: Paired differences $S_{\text{method}} - S_{\text{plain}}$ (nats), mean $\pm$ sample SD over five seeds. Superscripts count negative seed pairs. The populations and protocols are those of Table 13.

Evaluation set	ϵ	ΔS : R	ΔS : G	ΔS : RG
<i>Saved BatchNorm statistics</i>				
Train subset (10k)	0.1	$-0.264 \pm 0.045^{5/5}$	$-0.006 \pm 0.111^{3/5}$	$-0.301 \pm 0.045^{5/5}$
Train subset (10k)	0.25	$-1.454 \pm 0.237^{5/5}$	$-0.846 \pm 0.404^{5/5}$	$-1.741 \pm 0.355^{5/5}$
Full test set (10k)	0.1	$-0.218 \pm 0.041^{5/5}$	$+0.008 \pm 0.108^{2/5}$	$-0.239 \pm 0.043^{5/5}$
Full test set (10k)	0.25	$-1.250 \pm 0.246^{5/5}$	$-0.696 \pm 0.412^{5/5}$	$-1.496 \pm 0.355^{5/5}$
<i>BatchNorm recalibrated at each point</i>				
Train subset (10k)	0.1	$-0.057 \pm 0.014^{5/5}$	$+0.091 \pm 0.047^{0/5}$	$-0.064 \pm 0.024^{5/5}$
Train subset (10k)	0.25	$-0.250 \pm 0.059^{5/5}$	$+0.264 \pm 0.126^{0/5}$	$-0.291 \pm 0.099^{5/5}$
Full test set (10k)	0.1	$-0.013 \pm 0.008^{5/5}$	$+0.110 \pm 0.043^{0/5}$	$-0.011 \pm 0.021^{4/5}$
Full test set (10k)	0.25	$-0.084 \pm 0.057^{5/5}$	$+0.379 \pm 0.130^{0/5}$	$-0.093 \pm 0.102^{4/5}$

the random directions isotropic in relative coordinates: their block covariance includes a factor inverse to block dimension. Random-direction averages and extreme eigenvalues answer different questions.

Hessian–vector products are computed by differentiating the loss gradient, accumulated with correct sample weights. Lanczos uses full reorthogonalization and two deterministic starts for each of 160 problems (twenty networks, two probes, two fixed-statistics policies and two coordinate choices). Each requested eigenpair must satisfy an explicit residual criterion of the form

$$\|Hv - \lambda v\|_2 \leq 10^{-3} \max\{|\lambda|, 10^{-3}|\lambda_{\max}|\}, \quad \|v\|_2 = 1,$$

with H replaced by H_{rel} when appropriate. All 160 problems met the criterion for the two largest and the smallest algebraic eigenvalue, from both starts; they required 40–110 iterations. The largest recorded relative residual is 6.4×10^{-7} for the two leading eigenpairs and 9.95×10^{-4} for the minimum eigenpair. Residuals assess the numerical eigenproblem, not whether the checkpoint is a local optimum.

The evaluations use float32 with TF32 disabled and deterministic cuDNN settings. On 960 small-amplitude comparisons, float64 recomputation changes S by at most 4.0×10^{-7} nats. Repeated HVPs were identical in the recorded checks; the relative symmetry discrepancy was at most 5.7×10^{-6} , and float32–float64 HVP differences at most 4.1×10^{-4} . Dense pilot checks and classifier-only central finite differences support the implementation. Finite differences through all perturbed ReLU weights do *not* show uniform convergence over the tested steps. We therefore interpret the HVP as the automatic-differentiation local Hessian, not as a quadratic model valid over every plotted interval.

The smallest eigenvalue is negative in all 160 cases, and the corresponding checks include float64 Rayleigh quotients. Yet the CE gradient norms range from 1.88 to 7.37 on the selected parameters: the checkpoints are

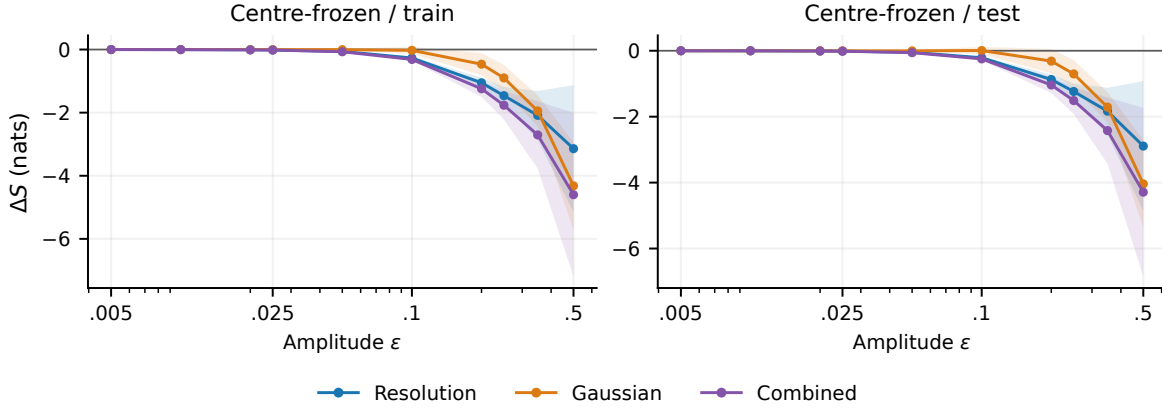


Figure 6: The centre-frozen policy, using the same conventions as Figure 1. R and RG have negative paired differences at every amplitude in every seed on both probes. G does not share that full pattern.

Table 15: Largest algebraic Hessian eigenvalue, mean $\pm$ sample SD over five seeds. CIFAR-10 / ResNet-20-BN / SGD, 30 epochs; CE on 1k-image probes and 268 336 convolution/classifier weights. Coordinate groups define different matrices; compare methods within rows.

BatchNorm policy	Probe	Plain	R	G	RG
<i>Ordinary parameter coordinates</i>					
Saved	Train	2382 $\pm$ 147	1444 $\pm$ 100	1281 $\pm$ 116	1148 $\pm$ 56
Saved	Test	3237 $\pm$ 324	1677 $\pm$ 77	1500 $\pm$ 120	1282 $\pm$ 59
Centre-frozen	Train	2354 $\pm$ 160	1438 $\pm$ 107	1275 $\pm$ 128	1135 $\pm$ 70
Centre-frozen	Test	3230 $\pm$ 296	1664 $\pm$ 70	1482 $\pm$ 104	1268 $\pm$ 69
<i>Block-relative parameter coordinates</i>					
Saved	Train	3818 $\pm$ 249	2308 $\pm$ 157	2101 $\pm$ 198	1859 $\pm$ 80
Saved	Test	5207 $\pm$ 528	2674 $\pm$ 139	2460 $\pm$ 207	2073 $\pm$ 88
Centre-frozen	Train	3775 $\pm$ 260	2297 $\pm$ 167	2090 $\pm$ 215	1838 $\pm$ 107
Centre-frozen	Test	5194 $\pm$ 480	2654 $\pm$ 126	2430 $\pm$ 179	2052 $\pm$ 107

not stationary points of these evaluation losses. These facts are compatible with visually bowl-shaped random slices. Neither a positive random slice nor a finite straight-line barrier certifies a local minimum or its basin.

D.5 Intermediate states, segments and PCA

Ten directions are evaluated at epochs 3, 6, 9, 12, 18, 21 and 30, at amplitudes 0.1 and 0.25, under both the state that produced the checkpoint and the final state. The recorded transitions additionally compare before/after states at fixed weights. R already has lower sensitivity in the first measured produced-state checkpoints; this does not establish what happened before epoch 3. For G, the saved-statistics ordering changes as filtering is removed, whereas pointwise-recalibrated sensitivity need not follow it. A lower loss under the state that produced the weights is common, but not universal: at G’s epoch-3 transition with recalibration, the next state has slightly lower loss in 1 of the five training-probe seeds and 3 of the five test-probe seeds. We avoid a blanket claim that the training state always minimizes fixed-weight loss.

Straight-segment evaluations use

$$\theta(\alpha) = (1 - \alpha)\theta_A + \alpha\theta_B, \quad B_{A,B} = \max_{\alpha \in \mathcal{G}} L(\theta(\alpha)) - \max\{L(\theta_A), L(\theta_B)\},$$

where $\mathcal{G}$ is a 51-point uniform grid in $[0, 1]$. They use the final intervention state and pointwise-recalibrated BatchNorm. The comparisons are plain-R, plain-G, plain-RG and R-RG, each at five seeds. The barriers are positive on both probes, but the R-RG barrier is not always higher than plain-R: it is lower in 3 of the five seeds on the training probe (seeds 1, 2 and 4) and in 1 on the test probe (seed 1). These are barriers on

sampled straight segments in the given parameterization. They do not exclude low-loss curved paths, as found by mode-connectivity methods (Garipov et al., 2018); no optimized connecting path was computed here.

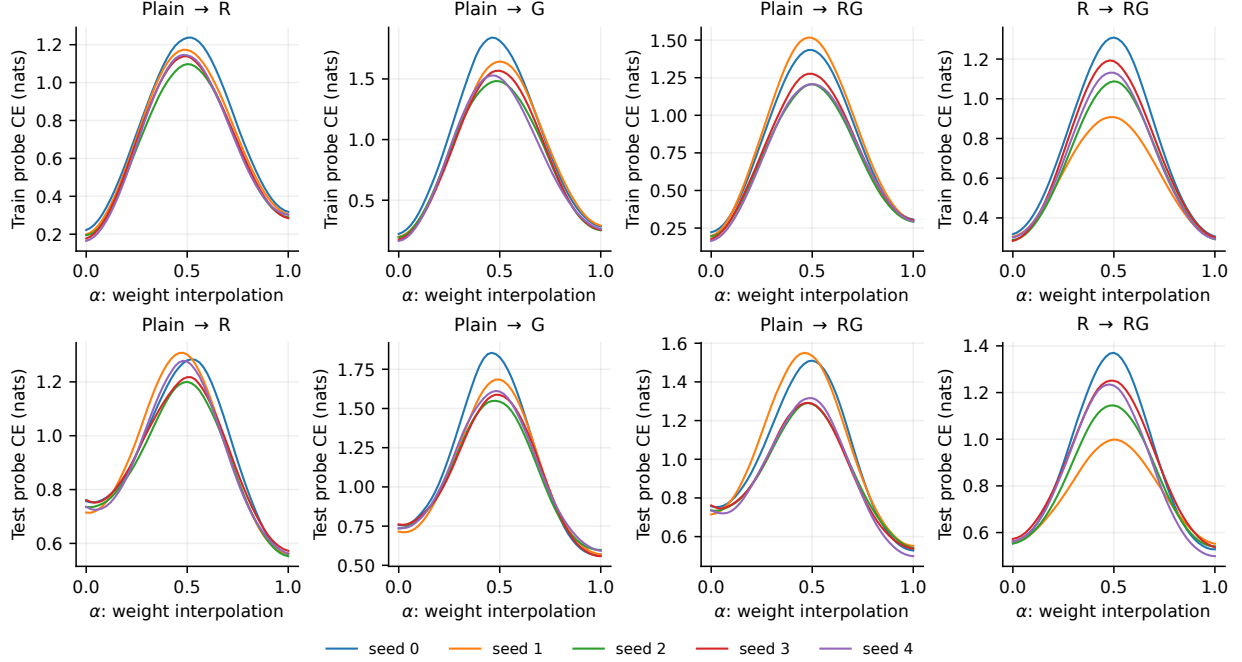


Figure 7: Measured straight-segment losses on the training and test probes, with recalibration at every point. Each thin line is one seed. Endpoints correspond to separately trained models, and the horizontal coordinate is interpolation in weights, not training time. Barriers do not establish disconnected basins.

PCA summarizes the recorded weight trajectory rather than selecting random directions (Li et al., 2018). In the shared four-method analysis, the first two components explain only 56.6–59.0% of the centred checkpoint variance across seeds; four explain 93.2–93.4%. Large individual projection errors remain in two dimensions. A checkpoint’s projected location on a plotted plane is consequently not the checkpoint itself, and the plane’s CE at that location is not its measured CE. Connecting recorded checkpoints also does not recover the unobserved optimization path between them. We retain these projections as diagnostics and do not use them as the main evidence for a geometric mechanism.

E Exploratory input-derivative measurements

A separate grid contains 28 models: four methods at three reference seeds, plus four methods at seed 0 under each of four changes (learning rate 0.0025 or 0.01, weight decay 10^{-4} or 2×10^{-3}). All use CIFAR-10 (50k training / 10k test images), ResNet-20-BN/ReLU, SGD and 30 epochs. These twenty method–recipe conditions are exploratory; the changed recipes have only one seed and are not recommended hyperparameters. All diagnostics concern final checkpoints.

E.1 Input Jacobians and finite logit changes

Let $x \in [0, 1]^{3 \times 32 \times 32}$ be an image in raw intensity units. Per-channel normalization is included inside $f_\theta(x) \in \mathbb{R}^{10}$, the vector of logits. With the model in eval mode, saved BatchNorm statistics and the final intervention state, we compute

$$J(x) = \frac{\partial f_\theta(x)}{\partial x} \in \mathbb{R}^{10 \times 3072}.$$

Ten backward passes give the rows of J ; singular values follow from its float64 10×10 Gram matrix. Norms average the first 1,000 images in the pinned training order, without class balancing. Units are logits per unit

Table 16: Input diagnostics in the separate reference grid: CIFAR-10 / ResNet-20-BN / SGD, 30 epochs, saved BatchNorm; mean $\pm$ sample SD over three seeds. Jacobian norms average 1k training images and use raw-intensity coordinates. Radius is in cycles per image. Bold accuracy means exceed Plain.

Method	Test accuracy (%)	$\mathbb{E}\ J\ _F$	$\mathbb{E}\ J\ _2$	Mean radius
Plain	75.93 $\pm$ 1.04	143.2 $\pm$ 1.8	78.7 $\pm$ 2.3	12.08 $\pm$ 0.33
Resolution	80.22 $\pm$ 0.01	100.0 $\pm$ 3.6	53.6 $\pm$ 2.0	12.63 $\pm$ 0.17
Gaussian	79.95 $\pm$ 0.04	74.8 $\pm$ 4.0	39.5 $\pm$ 2.2	11.01 $\pm$ 0.25
Combined	81.35 $\pm$ 0.09	78.1 $\pm$ 1.4	41.6 $\pm$ 0.8	11.46 $\pm$ 0.18

input intensity. Logit scaling can change these norms without changing classification decisions; they do not give the uniform bounds required by a margin argument such as Sokolić et al. (2017).

For the spatial Fourier profile, we apply an orthonormal two-dimensional transform to each Jacobian row, sum squared magnitudes over logits and channels, and group them by rounded radius. Energy fractions are normalized within each 250-image batch, then averaged over four batches. The energy-weighted mean radius is in cycles per image; this is a batch-normalized Jacobian spectrum, not a mean of per-image normalized spectra.

For each image, let $v_1(x)$ be a unit top right singular vector of $J(x)$. The finite input diagnostic is

$$D(\varepsilon) = \frac{1}{N} \sum_i \|f_\theta(x_i + \varepsilon v_1(x_i)) - f_\theta(x_i)\|_2, \quad \varepsilon \in \{0.05, 0.1, 0.25, 0.5, 1, 2, 3, 6, 12\}.$$

The singular direction is fixed at the original image. Inputs are not clipped to $[0, 1]$, so large perturbations leave the image domain. This logit displacement differs from the CE sensitivity S . At $\varepsilon = 0.5$, the ratio $D(\varepsilon)/(\varepsilon \mathbb{E}\sigma_{\max}(J))$ is about 0.32–0.47 in the reference runs: the local linear approximation is already inadequate.

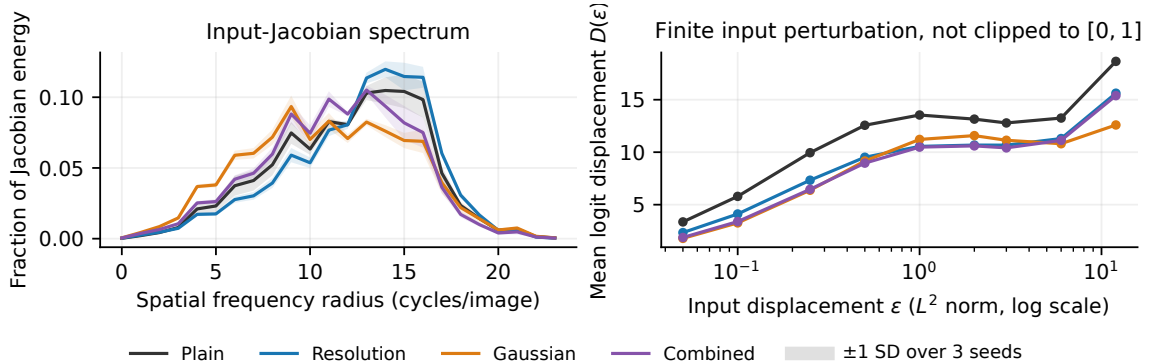


Figure 8: Input-diagnostic reference group. Left: Jacobian frequency profiles, mean $\pm$ SD over three seeds. Right: mean logit displacement along the top input singular direction, without clipping. R lowers the Jacobian norm but increases its mean frequency radius.

All three interventions lower the mean Jacobian norms, but R increases the mean frequency radius whereas G and RG decrease it (Table 16). These measurements therefore do not establish a common low-frequency explanation of improved accuracy.

E.2 Parameter trace and cross-condition analysis

The grid also estimates the Hessian trace of mean training CE on 5,000 images, with saved BatchNorm and all 269,722 parameters, including biases and normalization parameters. Five power-iteration components deflate a 64-draw Rademacher Hutchinson estimate (Hutchinson, 1989). Mean reference traces are 14 731 (Plain), 10 108 (R), 10 361 (G) and 8 761 (RG). The parameter subset, data and solver differ from Appendix D.4, so the two studies are not pooled.

Of 140 power-iteration estimates, 135 met a relative-change threshold of 10^{-4} within 40 iterations, including

all 28 leading estimates. This is not an eigenvector-residual certificate; the main Hessian comparison uses the residual-checked Lanczos results.

Table 17 gives the full twenty-condition grid. Exploratory correlations with test error, controlling training error, weakened when restricted to intervention conditions. Unequal replication, few conditions and joint dependence on method and recipe limit that analysis. It neither validates a predictor on held-out conditions nor establishes causality.

Table 17: All twenty input-diagnostic conditions: CIFAR-10 / ResNet-20-BN / SGD, 30 epochs. Each block changes one reference hyperparameter. Entries are means over n seeds; changed recipes have one seed and no estimated SD. CE uses the full training/test sets; Jacobian norms use 1k training images. The all-parameter trace uses 5k images. Bold accuracy means exceed Plain within the same recipe.

Method	n	Test acc. (%)	Train CE	Test CE	$\mathbb{E}\ J\ _F$	$\widehat{\text{tr}}(H)$
<i>Reference: $\eta = 0.005$, $w = 5 \times 10^{-4}$</i>						
Plain	3	75.93	0.209	0.755	143.2	14 731
Resolution	3	80.22	0.300	0.577	100.0	10 108
Gaussian	3	79.95	0.270	0.595	74.8	10 361
Combined	3	81.35	0.311	0.552	78.1	8 761
<i>Learning rate $\eta = 0.0025$</i>						
Plain	1	71.33	0.600	0.816	83.7	12 308
Resolution	1	74.08	0.606	0.741	69.9	9 863
Gaussian	1	75.52	0.507	0.706	60.0	11 541
Combined	1	76.76	0.533	0.668	63.8	9 473
<i>Learning rate $\eta = 0.01$</i>						
Plain	1	78.21	0.024	0.790	189.3	4 744
Resolution	1	82.95	0.092	0.546	135.6	7 356
Gaussian	1	81.93	0.124	0.563	81.6	7 245
Combined	1	83.88	0.158	0.487	100.1	6 525
<i>Weight decay $w = 10^{-4}$</i>						
Plain	1	73.58	0.256	0.793	147.6	12 536
Resolution	1	80.03	0.332	0.590	102.6	8 701
Gaussian	1	79.54	0.310	0.607	73.5	9 243
Combined	1	80.72	0.343	0.568	79.3	7 534
<i>Weight decay $w = 2 \times 10^{-3}$</i>						
Plain	1	77.23	0.146	0.697	132.2	22 648
Resolution	1	81.94	0.213	0.537	91.2	17 584
Gaussian	1	81.91	0.197	0.535	63.2	16 493
Combined	1	83.18	0.233	0.489	72.0	14 604

Validation and scope. All 28 re-evaluated test accuracies match their training-log endpoints, with CE discrepancies below 2×10^{-8} nats. The intervention runs have unusually narrow seed spreads (R: 80.21, 80.22, 80.22%). Initial-weight and data-order records distinguish the seeds, and trajectories diverge, but final-checkpoint hashes were unavailable to the audit. We keep this batch separate and make no claim of reduced seed variance.